

Neurolabs iOS SDK Integration Guide

Technical Integration Specification

Document ID: NLB-IOS-INTEGRATION-GUIDE

Version: v1.7.16

Date: 2026-10-09

Status: Partner integration reference

Owner: Neurolabs

1. Scope

This guide is for partner iOS apps integrating 'NeurolabsSDK' through Swift Package Manager.

2. Release History

No public API contract has been broken since v1.1.9. New options have been added with backwards-compatible defaults. Recommended defaults: strict shelf guidance with overlays disabled, 'liveQualityEnabled: true', 'autoCloseAfterCapture: true' for single-capture parity flows. 'NLCameraConfiguration.captureRange(...)' is the preferred helper for custom-camera flows that need a minimum capture count and a higher maximum cap. 'showsCapturedCountLabel' hides the 'N captured' chip in custom camera and multi-capture flow UIs.

v1.3.2

- Mixpanel ingestion routes to the EU region via 'NLANalyticsContext.resolveMixpanelEndpoint(fallback: .mixpanelEndpointEU)' at SDK construction; the 'NLMixpanelAnalyticsTracker' default itself stays on the standard US endpoint so partners reusing the class for their own US-region projects are unaffected. Endpoint can be overridden via the 'NLMixpanelEndpoint' Info.plist key or the 'NL_MIXPANEL_ENDPOINT' environment variable (env wins over plist).
- Mixpanel 'time' field is now serialized as a JSON number (epoch seconds). Previously 'time' was sent as a string, which Mixpanel rejected with '{"status":0,"error":"time field must be a number"}', silently dropping every event.
- 'NLCustomCameraView.saveAllCaptures' keeps the Done button responsive: the synchronous 'FileManager.removeItem' loop for staged-capture files moved into a 'Task.detached(priority: .utility)' block, and the Done label swaps to an inline 'ProgressView' while 'isPersistingCaptures == true'. Opacity remains at 1 during persist so the indicator stays visible.
- New 'NLOperationsCatalogClient' (in 'ProductAuditKit') for the operations product catalog, with 'BarcodeLookupIdentity' normalising UPC-E/EAN-8/UPC-A/EAN-13 to GTIN-14 end-to-end. 'NLOperationsCatalogError.invalidBarcode' / '.invalidSearchQuery' / '.malformedResponse' are thrown rather than crashing the host with 'precondition'.
- 'NLBDemoApp' gains an in-app settings sheet on 'CustomCameraDemoView' (gear icon, top-right). Sliders / steppers / toggles for every tunable knob, bindings wired to existing '@AppStorage' properties so saved values survive across launches.
- Mixpanel event identity hardened: per-install distinct id persisted in 'UserDefaults', '\$insert_id' (UUID4) on every event for server-side dedup, response body surfaced when 'debugLogging' is true.

v1.3.1

- Restrict barcode symbologies to retail formats (EAN-13, EAN-8, UPC-A, UPC-E) so the scanner stops emitting non-product codes during capture.

v1.3.0

- Angle-off warning threshold relaxed to 12 for less aggressive blocking during normal in-store motion.
- Multi-capture preview deletion: persisted thumbnails are removed when the matching capture is dropped.
- AVFoundation preview recovery hardened against backgrounding / interruption.

v1.2.8

- 'ProductAuditKit' test targets added (no public API change).

v1.2.6 / v1.2.7

- Product Audit (onboarding) MVP: serialise 'AVAssetWriter' finalize and 'OCRSmartRotationProvider' work via in-flight 'Task', off-actor keyframe JPEG writes, completedStepIDs reflect steps that actually ran, step transitions serialised via pending-transition queue.

v1.2.5

- New 'showsSequenceCounterLabel' option in the native capture API for toggling the on-screen sequence counter.
- Persistent custom-camera tuning in the demo app (precursor to the v1.3.2 settings sheet) - preferences survive across runs via 'UserDefaults'.
- AVFoundation preview recovery improved on capture failure.
- Strict-rejection preview freeze regression fixed.

v1.2.4

- Release build / Swift 6 archive blockers resolved.

v1.2.3

- 'NLCameraConfiguration.captureRange(...)' helper for bounded multi-capture flows.
- 0.5/1 lens switcher built on AVFoundation, default camera mode starts on the ultra-wide lens when available. Tap-to-focus, lens switch fallback covered by unit tests.
- Custom-camera startup, lens switching, and guidance responsiveness improvements; camera preview shown during warmup, capture button gated until ready.
- Default task routing config uses 'taskUUID'.
- Recommended custom-camera parity flow remains strict shelf guidance with overlays disabled, 'liveQualityEnabled: true', 'autoCloseAfterCapture: true' with a single required capture.

v1.2.2

- Analytics tracking + opt-out, partner/device context, Mixpanel defaults, Sentry integration with privacy-safe defaults.

v1.2.1

- Initial lens switcher, memory fixes, bottom-bar centering and UI polish.

v1.2.0

- Docs aligned to v1.1.9 requirements baseline; no SDK code change.

v1.1.9 (baseline)

- Init and per-session routing use 'taskUUID'.
- Native detector/model warmup enforced in 'init' and guarded in 'openCamera'.
- 'autoCloseAfterCapture: true' keeps preview enabled and closes after Save confirmation.

3. Requirements

- iOS 17.0+ (SDK package declares '.iOS(.v17)')
- Xcode 16.4+ (release toolchain baseline)
- Swift 5.9+ (Swift 6 toolchains supported)

4. Install (SPM)

Add package dependency (mobile-dist or direct distribution manifest) and link 'NeurolabsSDK'.

5. SDK Initialization + Warmup

Uploads use the **operations platform** with a **single key**: the 'apiKey' passed to 'init' seeds the operations credential automatically, and every capture routes through the resumable operations mission flow ('init-resumable chunked PUT complete' per image, 'submit' per session) against 'api.operations.neurolabs.ai'. No base URL or separate credential call is needed; use 'setOperationsApiKey(_:baseUrl:)' only to rotate the key or to point at a non-production host (staging/dev).

```
import NeurolabsSDK

final class CaptureCoordinator {
    let sdk = NeurolabsSDKCore(configuration: .init(
        apiKey: "<OPERATIONS_API_KEY>",
        debugLogging: true,
        taskUUID: "<DEFAULT_TASK_UUID>" // optional IR-task override (never a mission id)
    ))

    func prepare() {
        // SDK starts image pipeline preparation during init.
        // Optional: observe loading progress through delegate.
    }
}
```

6. Queue Management

```
Task {
    await sdk.setAutoSyncEnabled(true)
    await sdk.setWifiOnlyUploadsEnabled(false)
    await sdk.setDeletePhotosOnUploadSuccess(true)
    await sdk.setMaxQueueSize(200)
    await sdk.setUploadRetryCount(5)

    let status = await sdk.getQueueStatus()
    print("pending=\(status.pendingCount) failed=\(status.failedCount)")

    await sdk.flushQueue()
    await sdk.retryFailedUploads()
}
```

Inspecting the queue at launch

Before you inspect, withdraw or discard queued work at launch, wait for the queue to finish loading the records an earlier launch left on disk:

```
Task {
    await sdk.awaitQueueHydrated() // the whole pass; returns at once once it has run
    for item in await sdk.getQueuedItems() where item.sessionId == abandonedSessionId {
        await sdk.removeQueuedItem(item.id)
    }
}
```

- It returns once the whole pass has finished, also when the load failed (the queue then holds what it could read). After a re-`configure()` it also waits for the replaced instance's queue to stop, so what it reports is settled.
- A setter that rebuilds the queue (`setMaxQueueSize(_)`, `setUploadRetryCount(_)`, a new operations key or base URL) starts a new pass; the call waits for the queue that is current when it returns.
- On an instance that `configure()` replaced it returns at once. Call it on `'NeurolabsSDKCore.shared'`.

7. Open Custom Camera

```
import UIKit
import NeurolabsSDK

var capture = NLCaptureConfiguration()
capture.confidenceThreshold = 0.25
capture.iouThreshold = 0.45
capture.maxCaptures = 1
capture.guidanceMode = .strict
capture.showDetectionOverlays = false
capture.showCapturedRegions = false

let config = NLCameraConfiguration.captureRange(
    capture,
    minimumCaptures: 1,
    maximumCaptures: 5,
    cameraMode: .default,
    showCapturePreview: true,
    showPreviewInStrictMode: true,
    showsCapturedCountLabel: false,
    enableValidation: true,
    showAlignmentGuidance: true,
    enableARDetections: false,
    liveQualityEnabled: true,
    liveQualityFPS: 6
)

let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        // Custom post-processing hook for each accepted capture
        // return .performDefault -> SDK also enqueues upload
        // return .skipDefault -> app takes full ownership
        return .performDefault
    },
    onSaveAll: { captures in
        // Batch hook (multi-capture flow)
        return .performDefault
    }
)

sdk.openCustomCameraUI(
    configuration: config,
    handlers: handlers,
    onClose: {
        print("Camera closed")
    }
)
```

Recommended capture payload notes:

- 'liveQualityEnabled: true' is the key for pill/rotation/warning/error guidance behavior in the custom camera.
- Keep shelf capture mode with strict guidance and overlays disabled for the custom-camera guidance UI path.
- Use 'NLCameraConfiguration.captureRange(...)' to express a minimum capture requirement plus a larger maximum cap.
- Use 'showsCapturedCountLabel = false' to hide the count chip when you do not want 'N captured' shown.

- 'guidanceMode: .guidance' should only be used as a temporary fallback if a partner explicitly wants looser guidance than the parity path.

Camera-only integration

If your app presents 'NLCustomCameraViewController' without creating a 'NeurolabsSDKCore', register an operations credential for the detector download first. Without it (or a core) the standalone camera has no detector: capture works, but live detection, the shelf guidance cues and the overlay stay off, and the camera says "Detector unavailable. You can still capture." (reason 'no_credentials'), not "Detector downloading".

```
NLDetectorModels.setCredentials(apiKey: operationsKey) // production
// NLDetectorModels.setCredentials(authProvider: provider, baseUrl: stagingURL)
```

The call is idempotent (the same key again changes nothing, so a failure's retry wait still holds), a new key retries at once, and the most recent call wins, this one or a core's. A base that is not absolute https with a host is refused.

For a signed-in rep's bearer, follow this lifecycle:

- **At sign-in**, call 'setCredentials(authProvider:)' with one provider for that rep. You can call it again on each camera open, as long as you pass that same instance. A new provider per call refreshes each time, and the new bearer counts as a new key, which clears the retry wait.
- **On account switch**, call it with a new provider.
- **On sign-out**, make the provider's 'token()' throw. There is no unregister, and a capture session asks the last provider again while the model is not on the device yet.

The download needs about 195 MB free (3.5 times the model: the archive, its unpacked copy and the compiled model are on disk at once during the install). With less it fails with 'insufficient_storage' and is retried at the next capture session. After an install the SDK deletes older model versions. If the installed model will not load, the SDK deletes it and downloads it once more; if that copy fails too, the status is 'load_failed' until the app restarts. A server answer that is not the model (a captive portal's page, a body of the wrong length) is 'download_failed', retried after a minute, not 'sha_mismatch'.

8. WebView Bridge Example

The same capture-range options are available through the native bridge models:

```
let config = NLNativeCaptureConfig(
    guidanceMode: .strict,
    showCloseButton: false,
    maxCaptures: 5,
    allowManualFinish: true,
    minCapturesBeforeDone: 1,
    showsCapturedCountLabel: false,
    cameraMode: .default
)

neurolabsSDK.openNativeCaptureUI(config: config, sessionId: sessionId)
```

Use 'NLNativeCaptureOptions' with the same field names when the bridge payload is built from JS or other app code.

9. Multi-Bay Capture (Long Shelves)

Multi-bay capture handles shelves too long for a single photo: the user side-steps along the shelf and captures one overlapping photo per bay, the SDK guides the side-step, dedups the overlap on device, and reports merged product counts across bays in a single session result.

Enable it by setting 'NLCameraConfiguration.multiBay' (nil = feature off, the default):

```
var config = NLCameraConfiguration()
config.multiBay = NLMultiBayOptions(
    minBays: 2,                // minimum bays before the session can finish (default 1)
    maxBays: 4,                // maximum bays per session (default 8, hard cap 8)
    targetOverlapFraction: 0.30, // fraction of the previous bay kept visible (default 0.30)
    minDetectionConfidence: 0.5, // confidence floor for merge/count participation (default
0.5)
    onDeviceDedup: true,        // run overlap dedup at session finish (default true)
    generatePreview: true,       // compose the display-only stitched preview (default true)
    previewMaxDimension: 2048    // preview long-edge cap (default 2048, clamped 512...4096)
)

sdk.openCustomCameraUI(configuration: config, handlers: handlers)
```

On the WebView bridge the same options travel as 'NLNativeCaptureOptions.multiBay' ('NLNativeMultiBayConfig'): every field is optional and absent fields take the 'NLMultiBayOptions' defaults. The field names are camelCase and identical across iOS, Android, and Cordova.

Reading merged results:

- Native session result: 'NLCaptureSessionResult.multiBay' ('NLMultiBayResult?', nil unless the session ran multi-bay) carries 'bays: [NLBaySummary]', 'mergedProducts: [NLMergedProduct]', 'countsByLabel: [String: Int]', the stitched preview ('previewImageData' / 'previewImageFileUri'), and 'dedupTrusted'.
- Use 'NLMultiBayResult.productCount' for a product tally - it sums only 'sku_single' + 'sku_multipack'; 'countsByLabel' also carries price/promo/poster buckets, so summing every label over-counts products.
- Bridge / SDK-managed flows: the 'native_capture_result' payload carries 'multiBay: NLNativeMultiBayResultSummary' with 'countsByLabel', 'baysCount', 'dedupTrusted', and 'previewImageFileUri'. Preview bytes are intentionally not sent over the bridge; the file URI is present only when a stable spilled file exists.

Meaning of 'dedupTrusted':

- 'true' - counts and 'mergedProducts' are on-device deduped: each physical product instance is counted once across overlapping bays.
- 'false' - an adjacent-pair alignment broke, the dedup timed out, or 'onDeviceDedup' was disabled; counts are then the per-bay sum (an upper bound), not a deduped count.

Upload semantics:

- Each bay photo uploads as a normal capture through the operations queue (resumable mission flow, one image per capture).

- On 'stopSession' the mission submit attaches a compact merge summary ({bays, counts_by_label, dedup_trusted, sdk_version, ref_homographies}) as structured 'client_metadata.device_dedup' plus, transitionally, a 'notes' duplicate for older backends - so the backend can re-dedup authoritatively.
- SDK-managed flows ('openNativeCaptureUI', 'openCustomCameraUI', bridge-driven capture) record that summary automatically. Only hosts presenting their own custom capture UI must pass 'NLMultiBayResult.submitNotesJSON()' to 'setMultiBaySubmitNotes(_:forSession:)' before 'stopSession(_:)'.

Config-only mode:

- There is no partner-facing in-camera mode toggle. The Single Multi-bay capture-mode chip and sheet are internal demo/debug chrome gated behind 'NLSDKDebug.internalCameraToolsEnabled' (default 'false'; never enable it in partner builds). Partners control the mode purely via configuration: 'multiBay' set = multi-bay session, 'multiBay = nil' = single-bay session.
- The Neurolabs demo app enables that chrome only in local Debug builds; TestFlight/Release demo builds default to single-bay exactly like partner builds. The multi-bay feature itself is fully available in all builds via configuration - only the demo defaults changed.

10. Post-Processing Hooks

- 'onSave' receives each approved capture.
- 'onSaveAll' receives final batch in multi-capture mode.
- 'onUploadRequested' exists but default queueing is done in 'onSave' / 'onSaveAll' path.

```
let handlers = NLCustomCameraHandlers(
    onSave: { capture in
        MyPostProcessor.shared.enqueue(capture)
        return .skipDefault
    }
)
```

Photo deleted (experimental)

'onSave' / 'onSaveAll' only ever see the final set. To hear about each photo the rep deletes, set 'NLCameraConfiguration.onCaptureDeleted':

```
var config = NLCameraConfiguration()
config.onCaptureDeleted = { deletion in
    // deletion.captureId the same id onCapture / onSave saw
    // deletion.remainingCount captures still in the session, pending + accepted
    // deletion.source .review | .captureStrip | .bayPeek
    MyVisitLog.shared.photoDeleted(deletion.captureId, remaining: deletion.remainingCount)
}
```

- Fires exactly once per delete the rep makes, on the main actor, after the capture has left the session and its files are gone.
- 'source' is '.review' for the review sheet and the final review grid, and '.captureStrip' for the per-tile delete on the capture strip. '.bayPeek' is reserved for the AR per-bay peek, which has no delete on iOS today.

- It does not fire for the SDK's own discards: a retake, a strict-quality reject, a capture your 'onCapture' took with '.skipDefault', the camera closing, or the upload queue removing a photo (including delete-on-upload-success).
- Nil, the default, sends nothing. It is a stored property, not an 'init' parameter, so it is additive and ABI-safe. The Android SDK has the same contract under the same names.

11. Delegate/Event Callbacks

```
final class SDKDelegate: NeurolabsSDKDelegate {
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didEncounterError error: NSError, sessionId: String?, messageId: String?) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didChangeQueueStatus status: NLQueueStatus) {}
    func neurolabsSDK(_ sdk: NeurolabsSDKCore, didUpdateLoadingProgress progress: NLLoadingProgress) {}
}

sdk.delegate = SDKDelegate()
```

'neurolabsSDK(_:didProduceCaptureResult:for:sessionId:)' is optional (it has a default empty implementation). Since 1.7.13 'openNativeCaptureUI' calls it once for every capture it saves, after trying to queue it:

- 'success' is 'true' when the capture was queued, and 'message' is "Capture saved and queued for upload."
- 'success' is 'false' when it could not be queued. 'message' says why, and the same error also arrives on 'didEncounterError'.
- 'detections' are the capture's detections, whether or not they are uploaded ('sendDetectionsMetadata'). 'capturedRegions' is empty.
- It arrives after 'didEnqueue' for the same capture. A capture the queue already holds is not reported again.

The whole session's result still arrives once, at the end, on 'onSendNativeCaptureResult'. A camera you open yourself with 'openCustomCameraUI' reports through your own handlers, not this callback.

12. On-Device Recognition (v1.7.1)

Per-account, config-gated: your operations API key resolves YOUR org's recognition config and vector dataset - enabling an account is a backend flip, no app change. Every stage degrades gracefully; capture is never blocked by recognition.

Source consumers (SwiftPM products 'RecognitionBootstrap', 'RecognitionEngine', 'RecognitionInterface'):

```

import RecognitionBootstrap
import RecognitionInterface

let bootstrap = NLRecognitionBootstrap(configuration: .init(
    operationsApiKey: "<your operations key>",
    sdkVersion: NLConstants.sdkVersion
))
// Wrap the components at the SDK boundary (see NLRecognitionProviderBox docs):
if let c = await bootstrap.build() {
    sdk.setRecognitionProvider(NLRecognitionProviderBox(
        isReady: c.isReady,
        recognize: { pb, dets, size in
            await c.recognize(pb, dets.map {
                NLProviderDetection(label: $0.label, score: $0.score, bbox: $0.bbox, id: $0.i
d)
            }, size)
        })
    })
}
// bootstrap.status / onStatusChange "provider: ready (N items)" etc.

```

Short-lived session tokens. When your bearer is a session token rather than a fixed key, pass an auth provider and a stable identity instead:

```

extension MySessionTokens: NLRecognitionAuthProvider {} // token() + invalidate()

let bootstrap = NLRecognitionBootstrap(configuration: try .init(
    authProvider: sessionTokens,
    offlineIdentity: rep.id, // stable across tokens; never empty
    sdkVersion: NLConstants.sdkVersion
))

```

The bootstrap asks the provider for the bearer on every request. A 401 calls 'invalidate()' and retries once; a second 401 turns recognition off. The offline record is named after a hash of 'offlineIdentity', so a cold start with no token (the provider throws) still builds from the last sync. Use a new bootstrap when the rep changes.

Offline (stores with no signal). Each build that completes online saves the recognition config and the snapshot's signed manifest for your credential. A later build that cannot reach the Operations API (offline, timeout, DNS, captive portal, 5xx) builds from it for 7 days after the last full sync ('NLRecognitionBootstrap.offlineCacheTTL'), if the cached embedder and every shard pass their checks. 'bootstrap.offlineSnapshotDate' is then the sync time, and 'bootstrap.statusSummary' reads 'provider: ready (offline, N items, snapshot from YYYY-MM-DD)'. A 401/403 still turns recognition off and deletes the cached build; a "disabled" config stays off. Have the rep open the app online once a day so the 7 days restart. A build that ended unavailable is retried by a later 'build()' (after 60 s, or 1 h for a definitive failure) and, after a network failure, on reconnect; call 'build()' again when 'onStatusChange' reports 'ready'. The cache lives in 'Application Support/nl-catalog-snapshots' (excluded from backup).

The embedder (the pinned DINOv3-ConvNeXt-tiny export, 768-d) is downloaded by the SDK through 'GET /v1/device-models' with your operations key, checked against the sha256 pinned in source, compiled on the device and cached. Do not bundle it: the SDK never scans the app bundle for a model (decision M5). When the model can't be had (not entitled, offline, a failed download or compile) the build is skipped as 'embedder unavailable (<reason>'. 'embedderModelURL' is the only local override, for air-gapped or bench setups, and it must still be a 768-d ConvNeXt export.

Binary consumers: 'RecognitionInterface' / 'RecognitionEngine' / 'RecognitionEngineQdrant' ship as xcframeworks on neurolabs-mobile-dist from v1.7.1; the bootstrap itself is source-only this release - wire the chain per the 'NLRecognitionProviderBox' documentation.

Tap-to-product-card: captures expose per-detection matches via 'sdk.recognitionMatches(forCapture:)'; drop the SDK's 'NLDetectionInspectorView(photo:title:detailsSource:)' onto your review screen and feed it an 'NLProductDetailsSource' wrapping ProductAuditKit's 'NLProductDetailsResolver' (one-liner on the source type's docs). Offline or unresolved products fall back to a UUID + similarity row automatically.

13. Notes

- Per-session routing is supported via native capture config task UUID overrides.
- Ensure 'NSCameraUsageDescription' is present in app 'Info.plist'.